

Chio: A Peer-to-Peer Economy of Verifiable Work

The Chio Project

<https://github.com/bb-connor/arc>

October 2026

Abstract

An agentic web requires programs that can recruit collaborators, delegate work and spend resources across organizational boundaries. Those programs must be free to change their plans while remaining bound by the commitments they issue. We present Chio, a kernel for composing agentic work under the authority of its resource owners. Chio binds delegated capacity, a selected invocation, receiver-local admission, execution evidence and funded payment into one verifiable chain of commitments. Each receiver enforces its own capabilities and treaty conditions. A running program can add tasks within its original allocation; earlier work retains its terms, execution identity and continuation custody. Recorded earned payments remain collectible under their agreed terms after the parent fails. We give the protocol, a conditional preservation argument and a Rust implementation. In a composed local execution, a first result triggers a newly funded collaborator, the intermediary is killed after that collaborator earns payment, and the original claim is collected while earlier execution evidence remains unchanged. Chio makes programmable sovereignty an execution rule: owners control admission, programs compose through explicit commitments, and changes to the plan preserve obligations already incurred.

1 A kernel for the agentic web

An agentic program chooses its next steps while it runs. A result may reveal a new task, a new collaborator or a need to delegate part of the work. Across organizations, those choices touch resources with different owners: tools, data, execution capacity and money. The program needs to compose them without absorbing their owners into one administrative domain.

Consider an intermediary agent commissioned by a buyer to review an API. It assigns an initial survey, then uses the returned result to hire a specialist. The specialist works under its own operator's policy and may finish before the intermediary does. If the intermediary crashes, the specialist's completed work and earned payment must have a meaning independent of the lost process. Meanwhile, the buyer needs to know that adding the specialist neither exceeded its allocation nor re-authorized an earlier invocation.

The difficulty is preserving commitments as the plan changes. An allocation must not become two promises merely because two providers respond. A new graph version must not turn a consumed continuation into another attempt. A signed foreign request must still satisfy the receiving owner's policy. A refund must not rewrite an uncertain tool effect or consume funds already owed to another participant. These conditions span planning, authorization, execution and settlement. They must agree on which work they describe.

Chio supplies that boundary. Its kernel mediates agent tool calls through capabilities, guards, durable admission and signed receipts. This paper extends that substrate into an execution contract for evolving work: delegate bounded capacity, seal an exact invocation, admit it under the receiver's

authority, retain its execution identity, and discharge its financial obligation under the agreed acceptance rule. The planner chooses the work; the resource owners enforce the commitments.

The organizing principle is *plans may evolve while issued commitments retain their bounds and identity*. Before a selection is sealed, an agent can choose another provider. After sealing, it must resolve the original work under its original identity. A running swarm can add tasks within its fixed envelope, while existing tasks keep their allocations and continuation resources. An earned payment follows its own agreement even when its parent fails.

This is *programmable sovereignty*: an owner expresses conditions for using its resources, and a kernel applies those conditions before an effect crosses the owner’s boundary. Treaties make cooperation explicit without importing a foreign participant’s authority. Commitments give that cooperation a durable meaning after the planner changes or disappears.

The contribution is the composition of these rules at the native invocation boundary. We specify the authenticated bindings between delegated work, signed swarm plans, receiver-owned admission and funded claims; establish their preservation property under stated custody assumptions; and implement the complete path in Rust. One executable trajectory runs initial work, discovers and funds a collaborator, grows the graph, survives intermediary process loss, and collects the collaborator’s original earned claim.

The resulting system is peer-to-peer in its authority structure: participants negotiate work and enforce their own admission conditions. Allocation and settlement still have explicit authorities. Unlike Bitcoin’s shared spend history [1], Chio requires no single global history for all programs. Each scarce resource has an identified custodian, and every handoff binds the same work to that custodian’s decision.

2 Programmable sovereignty

A kernel for agentic operating systems must give an untrusted planner useful freedom while preserving the owner’s control over effects. In Chio, the kernel is the trusted mediator for tool access and its durable records. Host isolation and tool confinement supply the surrounding enforcement boundary.

An *owner* creates a bounded work allocation and qualifies the principals that may use it. A *holder* can subdivide its allocation and select a provider. A *receiver* issues the capability for its tool and decides whether to admit the invocation. A *payer* backs a payment obligation in a chosen settlement domain, and a *verifier* applies the agreed acceptance procedure. These are roles, not a requirement for six organizations. Couriers carry signed evidence without acquiring the authority of its signers.

2.1 Authority stays with the resource owner

A work agreement expresses what parties intend to exchange. Admission decides whether a particular invocation may use a particular receiver’s resources. The receiver loads trusted issuers, policy, treaty records and replay state from its own protected stores. Request metadata cannot install those authorities.

A bilateral treaty supplies the conditions under which the parties cooperate. The receiver checks the configured participants and their signatures over the same canonical statement, its permitted action, validity and continuation, and the references binding it to this invocation. These checks join the receiver’s ordinary capability, revocation and guard checks. A valid work permit, treaty or funded allocation satisfies its own part of admission; none substitutes for the other required decisions.

The distinction also governs change. An owner can refuse a fresh invocation under its current policy. Replaying an admitted operation resolves its existing history. A claim already earned in escrow follows the immutable terms under which the payer funded it. Sovereignty determines which commitments an owner enters; the commitment’s lifecycle determines how it is discharged.

2.2 Three kinds of protected state

State	Authoritative fact	Preservation rule
Allocation and plan	Which bounded work has been committed	Subdivision conserves capacity; growth retains old tasks and allocations
Native custody	Which invocation was admitted and what execution established	Replay resolves the original operation and consumed continuation
Settlement	Which funds are reserved, earned, paid or refunded	Each transfer follows its own allocation and recorded decision

Table 1: The records are linked by authenticated bindings. A transition in one domain cannot stand in for evidence owned by another.

A sealed allowance is not a deposit. An authenticated execution statement is not a payment decision. A financial refund is not proof that a tool had no effect. Keeping these facts distinct is what allows the domains to compose: each contributes a decision it is actually authorized to make.

2.3 Trust and failure model

Agents and couriers may be malicious. Messages may be duplicated, reordered, delayed or lost, and processes may crash between durable steps. Safety assumes unforgeable signatures, collision-resistant canonical digests, valid clock checks, and honest enforcement by the selected resource custodians. Each delegation root has one protected allocator; each declared swarm pool has one protected graph lineage. Their custodians serialize allocation and head changes. Each receiving key has one protected execution and budget domain. The rail serializes financial transitions and couples them to actual asset transfers.

These are local trust requirements. Cloning a signing key and its database into independently spendable forks, or provisioning two graph lineages against the same declared pool, violates them. Tools must be confined well enough for their declared effects and readers to describe the protected operation. The acceptance authority and settlement observation must also be trustworthy.

The implemented discovery step selects among previously enrolled principals. A new result can create work and select its receiver; it cannot authorize a new reader or install a new trust root. This paper’s Rust profiles instantiate delegation, additive graph evolution and funded claims, identified in the artifact as D1, S1 and F1. Appendix A fixes their concrete encodings and limits.

Verifiable work means work bound to an agreed check and authenticated evidence. The kernel’s receipt attests its own mediated execution; the selected verifier decides the agreed acceptance predicate. Their signatures make those statements attributable and checkable. The value of the work depends on what the predicate measures.

3 From authority to a work commitment

The work commitment is the unit of composition. It develops through an ordered set of decisions: allocate capacity, choose an offer, seal the invocation, back its payment, and admit it at the receiver. The implementation uses existing signed records for these decisions. Their authenticated bindings make them refer to one piece of work.

3.1 Delegate bounded capacity

A work slot contains

$$S = (i, h, E, A, b, c, t, d, Q),$$

where i is its identity, h its holder, E its permitted server/tool pairs, A its permitted readers, b its ceiling in currency c , t its expiry, d its remaining delegation depth, and Q its acceptance predicate. The owner creates a root through a trusted configuration API. A holder signs a subdivision naming its parent allocation and the complete child slot.

The allocator admits a child only when its effects and readers are subsets of the parent's, its expiry is no later, its depth is smaller, and its currency is unchanged. All child ceilings together must fit within the parent's ceiling:

$$\sum_{j \in \text{children}(S)} b_j \leq b_S.$$

An atomic transaction couples the new child with the parent's allocation counter. A slot that has children cannot execute; a selected slot cannot acquire children. Unused internal capacity remains available for further subdivision. This prevents the same capacity from authorizing both a container and its leaves.

An allocation digest binds the allocator's persistent namespace, the complete root and the complete slot. Equal local names at different honest allocators therefore identify different allocations. A child can specify a different predicate because it performs a different task. The parent's result still requires its own check.

3.2 Select, then seal

A provider signs an offer over the allocation and slot digests, its receiving key, the exact effect and input, a price ceiling and an expiry. The holder selects that offer together with the original request ID and the digest of the receiver-issued capability. Revision checks serialize replacements while the selection remains open.

Sealing freezes the selection. The allocator persists a signed dispatch permit before returning it. That permit carries the immutable slot and signed selection, allowing the receiver to authenticate the commitment without a live connection to the allocator. It relies on the allocator's protected state for exclusive allocation and on the receiver's own custody for execution.

The receiver compares the permit with the actual subject, receiver, capability, tool, input and request. Its native grant authorizes one exact invocation with monetary ceilings no greater than the offer. The receiver also checks validity and current local authorization. A caller cannot substitute a different provider or input while retaining the original consent.

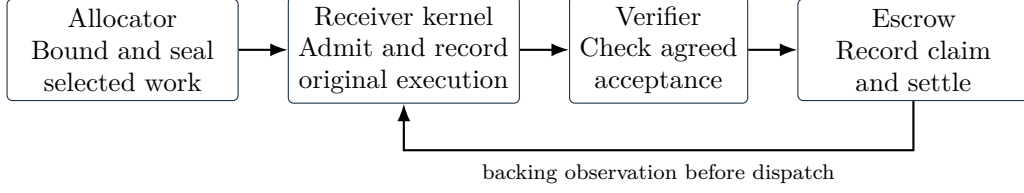


Figure 1: The exact work binding connects allocation, native execution and settlement. The receiver checks local authority and backing before dispatch. Its execution evidence feeds the agreed verifier; escrow enforces the recorded decision.

3.3 Back the obligation

Funded work adds an agreement that binds the complete final request, payer, beneficiary, verifier, asset, amount, settlement domain and deadlines. A qualified observation must establish that the agreement’s allocation holds the required funds. The escrow identifier includes its chain and contract domain; a new agreement cannot reuse another allocation’s reserve.

Each subcontract has its own backing. In the running example, the intermediary pays for the specialist from funds it already controls. Its expected payment from the buyer remains locked under a different agreement. This choice assigns the cost of a failed parent to the participant that hired the child, while preserving the child’s separately earned claim.

3.4 Bind the handoffs

The composed receiver checks that the delegated slot, native request and swarm task have the same identity. The swarm allocation names the delegated allocation digest and agrees on amount and currency. The sealed permit and swarm witness bind the same native capability, and the signed route targets this receiver.

The funded agreement then authenticates the complete request, including its delegation, graph and treaty references. Its financial allocation is a distinct identifier linked through that agreement. The hash chain has no cycle: the request contains the admission references; the agreement commits to the request; the escrow commits to the agreement.

Native signed bodies use canonical JSON and versioned domains [2]. Settlement decisions use the domain-bound EIP-712 encoding described in Section 4. Admission checks these relationships as well as each individual signature. Individually valid objects that refer to different work do not compose. The protected receiver profile installs the same checks before fresh work and recovery, so stripping request context or local configuration cannot select a weaker admission path.

4 Evolution, execution and recovery

A commitment gives a running program a stable object to preserve. The program can discover further work, but extending its plan must not enlarge an earlier permission, reassign a sealed invocation or reset a consumed execution right.

4.1 Grow the plan without replacing its history

Let G_i be the current signed swarm graph. An extension supplies the expected digest of G_i and a signed candidate G_{i+1} . The store first runs ordinary live-authority verification on both, then checks that the candidate retains the earlier graph envelope, tasks, edges, routes, witnesses, allocations and revocation epoch. New tasks must fit within the original depth, fan-out and pool bounds.

Each task has at most one allocation and one single-use continuation. An old continuation retains its identity and all fields except the graph digest and signature that bind it to the successor. A new task receives a new continuation. Signing a new graph therefore does not issue another attempt for an old task.

The installer atomically compares the expected head, archives the exact parent bundle and installs the successor. Two contenders using the same old head cannot both commit. A receiver resolves the graph version named in the original request, so old work remains interpretable after growth. Native custody owns the continuation by its stable identity across those versions. Fresh work still passes current capability, expiry, revocation and treaty checks.

In the running example, the survey's result creates the specialist task. The program seals the specialist's invocation, binds and funds its agreement, then installs the checked graph extension. The survey retains its original execution evidence. The intermediary can still execute its previously sealed request under the earlier graph.

4.2 Admit one invocation

The receiver checks local authority and the complete evidence binding before dispatch. It also verifies observed backing for the funded agreement. One native operation acquires the treaty and swarm continuation resources and binds the original financial allocation to its budget hold. The kernel records admission durably before invoking the tool.

For delegated work, the guard checks the exact output after transformations against the slot's predicate. Funded acceptance adds the agreement's verification procedure. The native execution statement binds the retained original request and output so that verification cannot silently consume another invocation's result.

A federated execution statement is exported from the original frozen admission provenance: treaty, participants, request and recorded admission time are checked against the retained outcome. Export supplies evidence of receiver-local execution. Final bilateral receipt delivery is a separate operation that may still require a remote co-signer.

4.3 Recover the fact that was established

A timeout describes a missing response. It does not establish whether the receiver dispatched, whether the tool had an effect, or whether a verifier recorded acceptance. Recovery therefore consults the resource owner's durable history.

Before sealing, revision checks permit another provider choice. After sealing, the original allocation and request remain bound even if a crash strands their permit. After native dispatch, recorded output can be recovered without another tool call. Where authoritative completion evidence is absent, the native state remains `OutcomeUnknownAfterDispatch`. Independent sibling work can continue from its own allocations.

The same rule governs settlement transactions. The reference implementation persists the exact signed transaction before broadcast. A restart reobserves or rebroadcasts those bytes; it does not choose a new nonce or substitute a different allocation. This preserves particular durable transitions. Arbitrary external tool effects still depend on the tool’s own effect and recovery model.

4.4 Earn, then collect

The funded lifecycle is

$$\text{Funded} \rightarrow \text{Submitted} \rightarrow \begin{cases} \text{Payable} \rightarrow \text{Paid}, \\ \text{Rejected} \rightarrow \text{Refunded}. \end{cases}$$

Undecided work can time out and refund under its agreed deadlines. An earned claim has no transition from Payable to timeout or refund.

The selected verifier’s decision binds the allocation, agreement, submission, beneficiary, asset and amount through typed signed data [3]. The escrow authenticates that decision. A claim is *earned* when an accepting decision is recorded under the selected settlement observation rule. An output or an unrecorded signature alone does not reach that state. An identical decision replay changes nothing; a conflicting replay fails.

Once the claim is earned, its beneficiary can withdraw without a new payer authorization. Parent completion, failure and refund concern a different allocation. In the running example, the specialist’s right to its reserved 100 survives the intermediary’s loss; the intermediary bears that subcontract cost. Its own refund does not establish that its unknown native invocation had no effect.

The owners commit in order: seal, bind the funded agreement, deposit backing, install graph growth, admit and dispatch, then record acceptance. There is no distributed transaction across those steps. A failure can leave capacity or funds committed until their existing lifecycle resolves them. Preserving that obligation is what prevents recovery from manufacturing a second promise.

5 The preservation property

The protocol has a single composition criterion: every allowed change must preserve the bounds and identity of existing commitments. Three invariants make this criterion precise.

5.1 Capacity is allocated once

Let B_D be a delegation root’s ceiling, b_v the ceiling of a leaf and U_v the unused remainder of an internal slot. Subdivision preserves

$$\sum_{\text{leaves } v} b_v + \sum_{\text{internal } v} U_v = B_D, \quad b_v, U_v \geq 0. \quad (1)$$

It moves capacity from a parent remainder into a child, without changing the sum. Internal slots cannot execute; selection and sealing change no ceiling. Declared effect and reader inclusion, decreasing expiry and decreasing depth extend transitively along the allocation tree. With the bound one-invocation capabilities and native charge enforcement, admitted leaf charges fit those ceilings.

For a swarm lineage, let A_i be the set of uniquely identified allocations in G_i , with fixed ceiling $b(a)$ and original declared pool B_G . Every checked extension satisfies

$$A_i \subseteq A_{i+1}, \quad b_{i+1}(a) = b_i(a) \ (a \in A_i), \quad \sum_{a \in A_{i+1}} b(a) \leq B_G. \quad (2)$$

A single protected lineage per declared pool, with a serialized head, establishes the domain of this bound. Because earlier allocations are retained, $\bigcup_{i=0}^n A_i = A_n$: the latest bound also covers every allocation ever installed. These are capacity bounds; deposited assets have a separate accounting invariant.

5.2 Funds follow their own allocation

For each accounted endowment u , fix deposited funds D_u with no further deposits into that accounting cohort. Let A_u be its available balance, L_u its locked funds, and P_u, R_u its paid and refunded outflows. Then

$$D_u = A_u + L_u + P_u + R_u, \quad A_u, L_u, P_u, R_u \geq 0. \quad (3)$$

Reservation moves funds from A_u to L_u ; payment or refund moves an allocation's locked funds once to the corresponding outflow. Each transition preserves the sum and nonnegativity. Returned cash remains an outflow in this cohort. A later deposit is accounted as another endowment; it neither resets these accounts nor releases their still-locked allocations.

An earned allocation remains in $\{\text{Payable}, \text{Paid}\}$. A transition on another allocation leaves it unchanged. Thus a parent refund cannot consume an earned child's reserve. Eventual withdrawal also requires the beneficiary's key, available evidence, transaction inclusion and a functioning asset.

5.3 Compose through the original invocation

Proposition 1 (Preservation under evolving work). *Assume the protected allocation, graph, receiver-custody and settlement domains of Section 2, the authenticated bindings of Section 3, and the transitions of Section 4. Begin with validly provisioned roots, verified initial graphs and funded accounts. Within those declared domains, every finite interleaving of valid subdivision, sealing, graph growth, native execution, recovery and settlement preserves the following properties:*

1. *Declared delegation and swarm capacity remain within their original bounds; paid plus refunded funds never exceed deposited backing.*
2. *A sealed invocation retains its selected receiver, capability, input, effect and request. Graph growth and replay grant it no additional native invocation right.*
3. *A recorded earned child claim retains its original terms and remains payable or paid through any parent failure, completion or refund.*

Proof. Initially, a new root has all its capacity in one leaf, a verified graph fits its declared pool, and a fresh funded endowment has $(A_u, L_u, P_u, R_u) = (D_u, 0, 0, 0)$. No execution right has been consumed or payment earned. The properties hold in this initialized state.

Induct over the interleaving. A subdivision changes only the parent's remainder and its new child, preserving (1). A selection changes no capacity. Sealing freezes the selected binding before evidence escapes.

A graph extension preserves every earlier allocation and checks (2) before committing its new head. The old continuation keeps its resource identity. Exact historical lookup preserves the artifacts used by an old request; a new graph signature cannot cause native custody to acquire that resource as a second invocation.

Admission compares the request with the sealed permit, native capability, graph task, route and treaty. The funded agreement authenticates that same request. A substituted object either fails the binding or requires a different authorized commitment. Native recovery resolves the original operation, hold and continuation claim. Refusal, expiry and revocation can prevent fresh work; they do not create a replacement invocation from the old commitment.

Financial transitions preserve (3). Their allocation identity confines each transition to its own reserved funds. A parent transition cannot change the child; a child transition after acceptance stays inside `{Payable, Paid}`.

These cases cover the allowed transitions. They remain valid when interleaved because additive growth retains the resources on which old execution depends, and each scarce resource is serialized by its own custodian. $\square$

The result explains why the construction needs no atomic transaction spanning all owners. An incomplete handoff can strand a commitment, but cannot use absence of a later record as permission to replace it. Safety is obtained by retaining the earlier fact and resolving it under its original authority.

This is a conditional protocol argument. The accompanying Lean development checks abstract financial conservation and earned-state lemmas; native tests exercise concrete admission and recovery boundaries. Their relationship to the Rust and Solidity implementations is tested rather than established by a machine-checked refinement proof.

Exclusive allocation is a necessary premise. Two receivers can each see a valid promise against one deposit while neither sees the other promise. Signatures alone cannot distinguish that execution from two individually affordable exchanges. A protected allocator, trusted ledger or settlement chain must supply the shared allocation fact. Chio makes that custodian explicit at each handoff.

6 Implementation and evidence

The Rust implementation connects the work contract to Chio’s existing delegation allocator, swarm-authority verifier, bilateral treaty admission, native continuation custody, receipt exporter and funded settlement adapter. The composed profile adds explicit binding checks at their handoffs. It uses the existing capability and escrow mechanisms.

The evaluation has two source epochs: current native qualification for the composed execution, and retained funded-model and comparison experiments at their recorded revisions. The artifact binds each to its source inventory, commands and outputs. All observed executions use author-controlled infrastructure; the composed run uses separate receiver keys and stores on one host, in-process treaty co-signers, a private development chain and mock tokens.

6.1 A program that grows after its first result

The executable begins with a declared envelope of 300 units and a graph containing a planning root, a survey task and an unstarted intermediary task. The survey inventories an OpenAPI document’s

authentication declarations and identifies an operation without required authentication. That actual result triggers selection of an enrolled specialist. Only then are its task, capability, sealed selection, funded agreement and 100-unit deposit created. The checked graph grows from three nodes to four. The intermediary invokes the specialist. A separately implemented Python checker accepts the bound result, and escrow records the specialist’s claim as **Payable**. Before the intermediary’s own return is captured, its process is killed with SIGKILL. Its funding later refunds while its native effect remains unknown. Parent and verifier rail signing are disabled. The specialist’s collector withdraws the original earned claim, then repeats collection without another invocation or transfer.

Work	Retained native outcome	Financial result
Survey	Completed; original evidence unchanged	Paid 100
Intermediary	Outcome unknown after dispatch	Refunded 100
Specialist	Completed; original claim retained	Paid 100

Table 2: One composed execution after graph growth, intermediary SIGKILL and recovery. Native observations and financial resolution retain distinct meanings.

Each receiver records one tool invocation and one native claim episode containing its treaty and swarm continuation resources. Operation, allocation, hold, authorization and physical claim identities remain unchanged through reopen and collection. Starting from 2,000 total mock units, final balances are buyer 900, intermediary 1,000, specialist 100 and escrow zero. Survey proceeds accrue to the intermediary, which separately funds the specialist.

The first collection report retains an unavailable remote co-signer diagnostic. A later open has no unfinished local operation to reconcile. Both reports retain the same execution and payment identities. The observed result is local completion and payment; final bilateral receipt delivery remains a separate boundary.

6.2 Adversarial and recovery checks

The rejection cases use authenticated adversarial inputs. A validly re-signed graph that changes an issued continuation is rejected. A sealed allocation cannot select another invocation. Missing treaty evidence, a foreign capability and a signed route to another receiver fail before funding observation or native work. A deliberate removal of the receiver-route check makes its regression fail for reaching funding observation.

Native tests also recover recorded tool output after process loss, allow independent sibling progress, preserve original unfinished work through graph extension, and reject a fresh request using a consumed continuation. Concurrent graph extensions yield one committed successor. These cases exercise the specific boundaries used in the preservation argument.

Current qualification completes 21 commands on an unchanged source inventory, including the composed execution and four existing earned-child crash and transaction-recovery cases. Six opt-in chain regressions are run explicitly in addition to the default standalone suite. Appendix B gives counts and source boundaries; the companion retains the command output and the initial failed runs.

6.3 What the comparisons establish

The financial state machine is also implemented using ordinary SQL transactions. It matches every compared state and balance along the retained representative traces. A competent conventional implementation therefore has the same tested monetary conservation and earned-child behavior.

The pinned upstream ERC-8183 implementation also pays a separately funded child after parent refund and rejects overcommitment of the available reserve. It additionally supports partial settlement, beyond this paper’s fixed-price profile. Chio’s explicit **Payable** state preserves recorded acceptance until a later withdrawal. These are lifecycle choices within expressive settlement systems, not evidence of an exclusive economic primitive.

The composed experiment establishes the implemented connection between evolving authority, native execution and an earned obligation. The authentication inventory is deliberately easy to check: it measures protocol behavior rather than an economic advantage in outsourcing that work.

7 Relation to existing mechanisms

Chio builds on established mechanisms for secure cooperation. Object capabilities support robust composition among mutually suspicious components [4]; Secure Linking connects certified component properties with proof-carrying authentication [5]. Proof-carrying code makes supplied evidence checkable against a consumer’s policy [6]. Chio applies receiver-owned checking to a work commitment whose allocation, invocation, execution history and payment are enforced by different custodians.

Contract Net organizes distributed work through announcements, bids and awards [7]. Agent Contracts combines task specifications, resource bounds and success conditions, including delegated budget conservation [8]. The additional boundary developed here is sealing that choice to an exact receiver-issued invocation and preserving it when the surrounding signed work graph grows.

Interface Automata and multiparty session types establish compatibility and communication properties within their respective models [9, 10]. The preservation result in Section 5 concerns authority, capacity, execution identity and earned obligations. Correct task decomposition and progress of an arbitrary agent program require further properties of its tasks and interaction model.

AgentBound’s deterministic governance and verifiable decision receipts overlap with pre-execution policy enforcement [11]. A2A supplies task, message, artifact and discovery interfaces [12]; AP2 specifies signed commercial and payment mandates [13]. OAuth token exchange permits deployment-specific trust and token semantics [14]. These mechanisms can carry rich application contracts. Chio’s contribution is the defined binding and enforcement path across the owners of the resources, rather than an expressiveness restriction on their message formats.

ERC-8183 is the closest economic construction: it defines escrowed agent jobs, providers, evaluator-approved completion and expiry refunds [15]. Section 6 and Appendix B compare the pinned implementations directly. ERC-8004 supplies discovery, reputation and validation interfaces [16]; such evidence can inform local policy while admission remains the receiver’s decision. Neither an agent registry nor a settlement ledger must serve as a universal authority over tool access.

Sagas use application-defined compensation for sequences of transactions [17]. Durable workflows and transactional outboxes likewise provide established techniques for retaining work and messages. Chio’s recovery rule makes the authority of each retained fact explicit: a compensating refund resolves money, while an uncertain tool effect retains its original status. Contingent payments and fair exchange offer stronger cryptographic constructions for particular verification and exchange

problems [18, 19]. The funded profile here uses an agreed verifier and settlement domain.

The architectural result is a common native boundary for these concerns. Changing a program’s plan preserves the authenticated commitments already admitted by its resource owners. This gives planners a defined operation for composing work, and implementers a concrete set of bindings and lifecycle rules against which to test another receiver.

8 Scope and deployment

The preservation property depends on the custodians identified in Section 2. A compromised allocator can overissue; forked custody can duplicate execution authority; a dishonest verifier can approve bad work; an asset issuer can prevent transfer. Deployment must protect those authorities and use a settlement observation appropriate to its failure model. A receipt authenticates the issuing kernel’s statement, not arbitrary remote computation.

The current evolution operation is additive. It preserves earlier allocations, permissions and continuation identities. Retirement, budget reclaim, receiver migration and changes to the graph’s policy envelope need explicit closure or fencing rules. A timeout alone is insufficient. Sealing can strand unused capacity, and delegation’s output and recovery checks can refuse completion after contract expiry. Retained graph history also carries storage and verification costs.

The implemented predicates check bounded properties of JSON output. The funded verifier may apply a richer agreed procedure, but useful work still depends on choosing the right specification. Child success does not establish parent success. Declared readers and effects require complete host mediation, and revocation cannot recall information already disclosed.

Availability is separate from integrity. Hashes detect changed bytes but cannot recover withheld evidence. Offline allocation permits still rely on the receiver’s local qualification and current capability checks. A payable claim preserves the right to withdraw; collection requires an available rail and eligible beneficiary. Until acceptance is recorded, verifier failure can leave completed work unpaid.

The evidence is local and deterministic. It establishes the reported execution and rejection behavior, with the model and implementation boundaries stated in Appendix B. It supplies no measured claim about public-chain finality, independent administration, hostile-load scalability, lower integration cost or profitable useful work. Those are deployment and economic properties to measure alongside the protocol’s safety conditions.

9 Conclusion

An agentic program must be able to outlive the process and organization that started it. Chio gives it a stable unit: a work commitment bound to delegated capacity, local admission, original execution and agreed settlement.

Planners discover new tasks and collaborators. Treaties retain each owner’s admission conditions. Durable custody attaches recovery to admitted execution. Funded claims preserve earned obligations after parent failure. The implementation composes these rules in one evolving execution.

Chio makes local authority and lasting commitments compatible with programs whose plans are still being written. Owners determine what may enter their domains. Agents compose the work within those bounds.

References

- [1] Satoshi Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System, 2008. URL <https://bitcoin.org/bitcoin.pdf>.
- [2] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, 2020. URL <https://www.rfc-editor.org/rfc/rfc8785>.
- [3] Remco Bloemen, Leonid Logvinov, and Jacob Evans. EIP-712: Typed Structured Data Hashing and Signing, 2017. URL <https://eips.ethereum.org/EIPS/eip-712>.
- [4] Mark Samuel Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006. URL https://worrydream.com/refs/Miller_2006_-_Robust_Composition.pdf.
- [5] Eunyoung Lee and Andrew W. Appel. Secure Linking: a Framework for Trusted Software Components (Extended Version). Technical Report TR-663-02, Princeton University, 2002. URL <https://www.cs.princeton.edu/techreports/2002/663.pdf>.
- [6] George C. Necula. Proof-Carrying Code. In *Proceedings of POPL*, pages 106–119, 1997. URL <https://homes.cs.washington.edu/~mernst/teaching/6.893/readings/necula-popl97.pdf>.
- [7] Reid G. Smith. The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980. doi: 10.1109/TC.1980.1675516.
- [8] Qing Ye and Jing Tan. Agent Contracts: A Formal Framework for Resource-Bounded Autonomous AI Systems. arXiv:2601.08815v3, 2026. URL <https://arxiv.org/html/2601.08815v3>.
- [9] Luca de Alfaro and Thomas A. Henzinger. Interface Automata. In *Proceedings of ESEC/FSE*, pages 109–120, 2001. URL https://web.cs.wpi.edu/~heineman/html/teaching_/CS562/p109-de_alfaro.pdf.
- [10] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty Asynchronous Session Types. In *Proceedings of POPL*, pages 273–284, 2008. doi: 10.1145/1328897.1328472. URL <https://mrg.cs.ox.ac.uk/publications/multiparty-asynchronous-session-types/>.
- [11] Anuj Kaul, Qianlong Lan, and Pranay Gupta. Behavioral Governance for Autonomous AI Agents: The AgentBound Framework. arXiv:2606.30970v2, 2026. URL <https://arxiv.org/html/2606.30970v2>.
- [12] A2A Project. Agent2Agent Protocol Specification, 2026. URL <https://github.com/a2aproject/A2A/blob/65dadbd9b900fe2d970e05416eb432a91347cd2a/docs/specification.md>. Source pin 65dadbd9b900fe2d970e05416eb432a91347cd2a.
- [13] AP2 Contributors. Agentic Payment Protocol, version 0.2, 2026. URL <https://ap2-protocol.org/ap2/specification/>. Source pin and retrieval hash in the artifact’s sources.json.

- [14] Michael B. Jones, Anthony Nadalin, Brian Campbell, John Bradley, and Chuck Mortimore. OAuth 2.0 Token Exchange. RFC 8693, 2020. URL <https://www.rfc-editor.org/rfc/rfc8693>.
- [15] Davide Crapis, Bryan Lim, Tay Weixiong, and Chooi Zuhwa. ERC-8183: Agentic Commerce, 2026. URL <https://eips.ethereum.org/EIPS/eip-8183>. Draft, source snapshot 365b4c02879f3e882b91281d42b4f57b406205e9.
- [16] Marco De Rossi, Davide Crapis, Jordan Ellis, and Erik Reppel. ERC-8004: Trustless Agents, 2025. URL <https://eips.ethereum.org/EIPS/eip-8004>. Draft, source snapshot 365b4c02879f3e882b91281d42b4f57b406205e9.
- [17] Hector Garcia-Molina and Kenneth Salem. Sagas. Technical Report CS-TR-070-87, Princeton University, 1987. URL <https://www.cs.princeton.edu/techreports/1987/070.pdf>.
- [18] Matteo Campanelli, Rosario Gennaro, Steven Goldfeder, and Luca Nizzardo. Zero-Knowledge Contingent Payments Revisited: Attacks and Payments for Services. Cryptology ePrint Archive, 2017/566, 2017. URL <https://eprint.iacr.org/2017/566>.
- [19] Stefan Dziembowski, Lisa Ekey, and Sebastian Faust. FairSwap: How to Fairly Exchange Digital Goods. Cryptology ePrint Archive, 2018/740, 2018. URL <https://eprint.iacr.org/2018/740>.

A Reference profiles and encodings

The artifact names three existing profiles: D1 for dynamic delegation, S1 for additive swarm evolution and F1 for funded work. The composed receiver joins their original records; it does not replace them with another wire format.

A.1 Delegation records

A D1 allocation digest binds the persistent allocator namespace, immutable root slot and target slot. Each holder-signed subdivision binds the parent’s allocation digest. The receiver’s offer binds the allocation, complete slot, receiver, effect, input, price ceiling and expiry. The holder’s selection includes the complete signed offer, capability digest, request ID and expected selection revision. The allocator-signed permit carries the frozen slot and selection. Signed domains are versioned; the qualified records use v2.

The original receiver envelope contains a slot ID, payload and permit. Its input digest binds the slot ID and exact payload. In the composed layout, the permit is carried in the governed request context and the payload is the entire original tool-argument object. The receiver selects the installed layout; the caller cannot downgrade it. The enclosing permit has its own signature and is excluded from the input digest to avoid a hash cycle.

Canonical numbers are bounded by $2^{53} - 1$, delegation depth by 32, children per slot by 64, slots per root by 4,096 and envelopes by 64 KiB. Acceptance is a conjunction of 1–16 JSON-pointer equalities or inclusive integer ranges. The one-invocation native capability is issued by the receiving kernel and names the exact subject, server and tool. Per-call and total monetary ceilings must be no greater than the signed offer, in the same currency.

The D1 offer specifies accepted-output pricing. The installed guard rechecks the binding before dispatch and validates the exact JSON value after output transformations. A qualifying predicate rejection withholds output and uses the qualified reversible-hold path for a zero-charge denial. The executed invocation remains consumed. Native capability validity and revocation remain independent requirements.

Sealing freezes the selection, then persists its signed permit before returning it. A crash between those durable steps can strand capacity. When a stored permit is available, a retry returns its original issuance time and signature. There is no post-seal replacement or automatic reclaim operation.

A.2 Swarm and receiver records

S1 retains the graph envelope, existing nodes, edges, joins, routes, witness chains, allocations, pool and revocation epoch. Existing continuations retain every field except the new graph digest and signature. New tokens and allocations name new tasks; each task has at most one of each.

The SQLite installer verifies both bundles at store-owned time and commits the head transition and parent archive together. Historical lookup checks the graph ID, signed graph digest, full bundle digest and strict encoding. Other stores need equivalent protected serialization and historical binding to implement the same profile.

The composed receiver pins its allocator, witness, treaty peer, program and runtime profile in protected deployment policy. Each open installs the runtime hook and delegation guard before reconciliation. The existing runtime database must remain present; losing it cannot provision an

empty continuation domain. The funded journal’s policy hash prevents removal of the composed configuration.

Bilateral admission uses the existing signed treaty, permission antecedents, invocation and continuation records. Such antecedents establish permission; completed-work evidence comes from the retained native outcome. Execution export revalidates that outcome and its frozen federation provenance under the original participant identities and admission time. It grants no new dispatch, settlement or remote-completion authority.

A.3 Funding and deadlines

F1’s native agreement authenticates the full final request, original receiver authority, checker context and settlement terms. The current workload uses 100 mock units per native agreement. The artifact-only F1 agreement is a different concrete schema; the companion maps both explicitly.

Let $t_s < t_c < t_r < t_f$ be submission, challenge, resolution and refund boundaries. Submission is allowed by t_s , first verifier decision recording when $t_c < t \leq t_r$, and undecided refund when $t > t_f$. The interval before decision is a timing boundary; the profile has no public challenge court. An available failed check yields rejection; missing required output or custody yields no eligible decision.

Acceptance is a domain-bound EIP-712 decision over the allocation, agreement, submission, decision digest, acceptance bit, beneficiary, asset and amount. The reference escrow has no later expiry for an earned withdrawal. Its administrative pause and verifier-registry updates govern new funding, without rewriting an existing claim. This fixed-price profile has no partial-settlement entrypoint. Fees, disclosure loss and opportunity costs fall outside its reserved-unit bound unless separately provisioned.

B Artifact and comparison details

The companion `ARTIFACT.md` gives executable commands, source pins, dependency locks and the mapping from protocol objects to implementation. The claim register records assumptions and counterexamples for each claim. Current native evidence and historical experiments retain separate source inventories. The PDF build verifies the derived results and frozen artifacts.

B.1 Current native qualification

The current campaign records 21 terminal commands, 36,556 unchanged source inputs and 48 hashed outputs. Rust pass counts by boundary are 54 workflow, 10 native delegation labels, five existing native composition, 453 swarm/runtime, 11 execution-export, 25 durable SQLite and 91 default standalone cases. The delegation count includes its subprocess worker. One pre-existing workflow doctest is ignored. The default standalone run ignores six opt-in chain cases; a separate run passes all six against the identical source inventory, exercising 97 standalone cases in total.

The campaign includes three Node wire/pin tests, 13 artifact tests, the evolving funded execution, four existing earned-child crash subcases, four strict Clippy commands, three format commands, the delegation example and the funded binary build. This qualifies the affected local boundaries, not the full workspace or a production deployment.

The suite outputs identify the tested bindings and recovery cuts. Intended negative controls and initial development failures remain distinct from the terminal passing results.

B.2 Retained financial model and implementation checks

Experiment	Result	Scope
Finite claim graph	5,508 states	Two prefunded edges and nine timestamps
Transition checks	132,192	Selected role, deadline and financial transitions
Contract suite	143 passing	Private-chain bytecode; 118 representative traces
SQL comparator	312 matched steps	Same representative traces; trusted sequential store
Upstream ERC-8183	76 passing	Unmodified suite at the pinned source
Paired escrow cases	20 passing	Same asset, fees, capital and evaluator trust
Lean specification	7 results	Abstract conservation and earned-state lemmas

Table 3: Historical results retain their original source identities. Their counts are not attributed to the changed native implementation.

The finite graph begins with a three-unit parent and a two-unit child. It explores submissions, decisions, payment, refund, expiry and execution uncertainty within that alphabet. The Solidity runner replays 118 representative financial traces, rather than all 132,192 transitions. It compares state, commitments, decisions and actual balances. A deliberately weakened model permits an earned claim to refund; the explorer finds the counterexample. An in-memory mutation of the escrow produces the corresponding intended test failure.

The Lean specification proves conservation over finite account traces, paid-plus-refunded bounds, persistence of earned states, and noninterference between distinct allocation entries. The native binding and graph-growth arguments are not mechanized refinements of the Rust code. These different forms of evidence support their stated boundaries rather than a proof of the implementation as a whole.

B.3 Matched alternatives

The ordinary SQL comparator implements the same monetary transitions without Chio kernel code. It matches 312 non-time steps over the 118 representative traces, with additional tests for child collection, allocation exclusivity and unknown execution. A conditional-backing experiment also compares a structural bound with an independent exhaustive oracle over 4,095 finite profiles. A conventional allocator obtains the same capital saving from selecting among exclusive alternatives before dispatch.

Upstream ERC-8183 is pinned to commit `142e669c1f`. Its 76 unchanged tests and 20 added comparison tests use equal mock assets, zero fees, capital and evaluator trust. Both implementations pay a separately funded child after parent refund, reject two 60-unit jobs funded from 100, and deny spending an unsettled parent reserve. The upstream implementation also pays an approved 60-unit partial claim from a 100-unit job and refunds the remaining 40 after rejection.

The lifecycle differences are concrete. Chio records acceptance before later withdrawal, applies a resolution cutoff, and preserves earned withdrawal through its funding pause or allowlist changes. The pinned upstream completion pays immediately; after its expiry grace period, completion can still win if refund has not executed. Its refunds are blocked while paused. These observations concern the pinned implementations, whose specifications and extension mechanisms admit other choices. No performance or security ranking is inferred.

The historical security-integration record binds its native runs to their earlier v2/v3 source inventories. Current composed qualification remains separate.